Simplified Guide to JWT Authentication with Spring Boot

Introduction:

Securing your applications is paramount in today's digital landscape. One robust approach is JWT (JSON Web Token) authentication. It offers a secure way to verify user identities. In this guide, we will walk through implementing JWT authentication in a Spring Boot app, using a simplified yet effective methodology. We'll cover controllers, services, configurations, and repositories, ensuring you're well-equipped to enhance your app's security.

Step 1: Setting Up Your Spring Boot Project Begin by creating a new Spring Boot project or utilizing an existing one. Expedite the process by using Spring Initializr, which sets up essential dependencies like Spring Web, Spring Security, and Spring Data JPA.

```
<!-- Include necessary dependencies in your pom.xml file -->
<dependencies>
    <!-- Spring Web for creating web APIs -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <!-- Spring Security for robust authentication and authorization -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <!-- Spring Data JPA for streamlined database interactions -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
   <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt</artifactId>
    <version>0.9.1</version>
  </dependency>
    <!-- Other dependencies... -->
</dependencies>
```

Step 2: Crafting User Entity and Repository Design a `User` class encompassing attributes like `id`, `username`, and `password`. Develop a `UserRepository` interface to facilitate smooth user data management.

```
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String username;
```

```
    private String password;
    // Getters, setters...
}

public interface UserRepository extends JpaRepository<User, Long> {
    User findByUsername(String username);
}
```

🔒 **Step 3: Configuring Spring Security** Create a `SecurityConfig` class extending `WebSecurityConfigurerAdapter`. Override `configure(HttpSecurity http)` to establish security rules and manage JWT authentication.

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    @Autowired
    private UserDetailsService userDetailsService;

    @Autowired
    private JwtUtil jwtUtil;

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http.csrf().disable()
            .authorizeRequests()
                .antMatchers("/api/auth/**").permitAll()
                .anyRequest().authenticated()
            .and()
            .addFilter(new JwtAuthenticationFilter(authenticationManager(),
jwtUtil))
            .addFilter(new JwtAuthorizationFilter(authenticationManager(),
jwtUtil, userDetailsService));
    }

    // Additional configurations...
}
```

👤 **Step 4: Implementing UserService** Develop a `UserService` interface with methods to load a user by username and save a new user. Implement `UserDetailsService` to retrieve user details from the database.

```
@Service
public interface UserService extends UserDetailsService {
    UserDetails loadUserByUsername(String username);
    void saveUser(User user);
}

@Service
public class UserServiceImpl implements UserService {
    @Autowired
```

```
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
        User user = userRepository.findByUsername(username);
        if (user == null) {
            throw new UsernameNotFoundException("User not found: " + username);
        }
        return new org.springframework.security.core.userdetails.User(
            user.getUsername(),
            user.getPassword(),
            new ArrayList<>()
        );
    }

    @Override
    public void saveUser(User user) {
        userRepository.save(user);
    }
}
```

🔐 **Step 5: Generating and Validating JWT Tokens** Create a `JwtUtil` class to generate and validate JWT tokens.

```
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import org.springframework.stereotype.Component;

import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

@Component
public class JwtUtil {
    private final String SECRET = "your-secret-key"; // Replace with a secure
secret key
    private final long EXPIRATION_TIME = 900_000; // 15 minutes

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = extractAllClaims(token);
```

```
        return claimsResolver.apply(claims);
    }

    private Claims extractAllClaims(String token) {
        return
Jwts.parser().setSigningKey(SECRET).parseClaimsJws(token).getBody();
    }

    public String generateToken(String username) {
        Map<String, Object> claims = new HashMap<>();
        return createToken(claims, username);
    }

    private String createToken(Map<String, Object> claims, String subject) {
        return Jwts.builder()
                .setClaims(claims)
                .setSubject(subject)
                .setIssuedAt(new Date(System.currentTimeMillis()))
                .setExpiration(new Date(System.currentTimeMillis() +
EXPIRATION_TIME))
                .signWith(SignatureAlgorithm.HS256, SECRET)
                .compact();
    }

    public boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    public boolean validateToken(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return (username.equals(userDetails.getUsername()) &&
!isTokenExpired(token));
    }

    // Additional utility methods...
}
```

🔑 **Step 6: Authentication Controller** Design an `AuthController` class to handle user registration and authentication.

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {
    @Autowired
    private AuthenticationManager authenticationManager;

    @Autowired
    private UserService userService;

    @Autowired
```

```
    private JwtUtil jwtUtil;

    @PostMapping("/register")
    public ResponseEntity<String> registerUser(@RequestBody User user) {
        userService.saveUser(user);
        return ResponseEntity.ok("User registered successfully!");
    }

    @PostMapping("/login")
    public ResponseEntity<String> loginUser(@RequestBody AuthenticationRequest
request) {
        try {
            authenticationManager.authenticate(
                new UsernamePasswordAuthenticationToken(request.getUsername(),
request.getPassword())
            );
        } catch (BadCredentialsException e) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Invalid
username or password");
        }

        UserDetails userDetails =
userService.loadUserByUsername(request.getUsername());
        String token = jwtUtil.generateToken(userDetails);

        return ResponseEntity.ok(token);
    }
}
```

🔍 **Step 7: Implementing JwtAuthenticationFilter** Create a `JwtAuthenticationFilter` class to handle JWT authentication and authorization for each request.

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    @Autowired
    private UserDetailsService userDetailsService;

    @Autowired
    private JwtUtil jwtTokenUtil;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
HttpServletResponse response, FilterChain filterChain)
            throws ServletException, IOException {
        final String authorizationHeader = request.getHeader("Authorization");

        String username = null;
        String jwtToken = null;

        if (authorizationHeader != null && authorizationHeader.startsWith("Bearer
")) {
```

```
            jwtToken = authorizationHeader.substring(7);
            try {
                username = jwtTokenUtil.extractUsername(jwtToken);
            } catch (Exception e) {
                // Handle token extraction/validation errors
                System.out.println("Error extracting username from token: " +
e.getMessage());
            }
        }

        if (username != null &&
SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails userDetails =
userDetailsService.loadUserByUsername(username);

            if (jwtTokenUtil.validateToken(jwtToken, userDetails)) {
                UsernamePasswordAuthenticationToken authenticationToken = new
UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());

                authenticationToken.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));


SecurityContextHolder.getContext().setAuthentication(authenticationToken);
            }
        }

        filterChain.doFilter(request, response);
    }
}
```

Conclusion You've successfully implemented JWT authentication in your Spring Boot app! Your application now boasts heightened security, ensuring only authorized users access sensitive resources. Remember, security is an ongoing journey, so keep yourself informed about best practices and continuously enhance your app's defenses. Happy coding and stay secure!